O Duelo de Contextos (Processos vs. Threads)

1. Objetivo

O objetivo deste trabalho é realizar um experimento exploratório comparando o overhead de criação, o custo de comunicação e a consistência de dados entre Processos (padrão POSIX fork) e Threads (padrão POSIX pthreads) em ambiente Linux ou macOS.

2. Instruções Gerais

- Formação: Grupos de 2 a 4 pessoas.
- Entrega: Link de um repositório público no GitHub contendo:
    1. Código-fonte em C.
    2. Um Makefile para compilação.
    3. Relatório em Markdown (README.md) com os resultados.
- Ambiente: O trabalho deve ser executado obrigatoriamente em ambiente Unix-like (Linux ou macOS).

3. O Problema: O Contador Compartilhado

O grupo deve implementar um programa que incremente um contador global até o valor de 1.000.000.000 (um bilhão), distribuindo o esforço entre N unidades de execução (onde N = 2, 4 e 8).

Parte A: Implementação com Threads

1. Criar N threads que dividem o trabalho de incrementar a variável global.
2. Experimento T1: Executar sem nenhum mecanismo de sincronização.
3. Experimento T2: Executar utilizando pthread_mutex.

Parte B: Implementação com Processos

1. Criar N processos (via fork) que dividem o trabalho.

2. Para a comunicação (IPC), deve-se utilizar Memória Compartilhada (shmget / shmat).
3. Experimento P1: Executar sem sincronização (observar comportamento).
4. Experimento P2: Executar utilizando Semáforos (sem_open) para proteger a memória.

4. Requisitos do Relatório

O relatório deve conter:

1. Assinatura do Hardware: O resultado do comando *lscpu* (Linux) ou *sysctl -a | grep hw.ncpu* (macOS).
2. Tabela de Tempos: O tempo real de execução (usando o comando time) para cada cenário (T1, T2, P1, P2) variando o número de N (2, 4, 8).
3. Análise de Corrupção: No experimento T1 e P1, qual foi o valor final do contador? Explique por que ele não chegou em 1 bilhão e como a variação do hardware do grupo influenciou o erro.
4. Gráfico de Escalabilidade: Um gráfico (pode ser gerado em Python/Excel) mostrando o tempo de execução vs. número de trabalhadores (N).

5. Critérios de Avaliação

- Código (40%): Organização, uso correto das APIs POSIX e tratamento de erros simples.
- Análise Empírica (40%): Veracidade dos dados apresentados e correlação com o hardware utilizado.
- Conclusão (20%): Capacidade de explicar qual modelo (Processo ou Thread) teve o maior overhead de criação e qual foi mais eficiente na comunicação.

Nota: Repositórios sem as medidas de tempo reais ou com códigos que não compilam serão desconsiderados.